

SIF x OpenClaw Integration Architecture

Amendment A31.1: Auto-TDD Validation Layer

Integrating Issue-to-Validation Decomposition into the Sovereign Intelligence Runtime

Version	Status	Author	Date	Classification	Parent
1.0.0	DRAFT	Daniel Shamir / SIF Architecture	2026-02-20	SIF Amendment Series	A31 (OpenClaw Integration)

Source Paper: Ahmed et al., "TDD-Bench Verified: Can LLMs Generate Tests for Issues Before They Get Resolved?" (IBM Research, arXiv:2412.02883, Dec 2024)

1. Executive Summary

This sub-amendment (A31.1) integrates findings from IBM Research's TDD-Bench Verified and Auto-TDD pipeline into the SIF x OpenClaw architecture defined in A31. The core contribution is threefold: (1) a validation gate for the Archetype-to-Skill bridge that ensures generated skills actually implement their source archetype, (2) an adequacy scoring framework adapted from code coverage metrics to measure the fidelity of Cognitive Archaeology extractions, and (3) a new PIK (Project Intelligence Kernel) pattern called Issue-to-Validation Decomposition that generalizes the Auto-TDD pipeline into a reusable reasoning primitive across all SIF subsystems.

The Auto-TDD paper demonstrates that LLMs can generate tests from issue descriptions *before* code is written, achieving 23.6% fail-to-pass rate with GPT-4o and coverage adequacy above 0.91 for successful tests. These metrics and pipeline patterns directly enhance five sections of A31 and introduce two new components (Section 4.14 and Appendix B).

Key Insight: Auto-TDD's 3-step decomposition (localize -> contextualize -> generate) is isomorphic to SIF's pointer graph traversal (activate -> assemble -> synthesize). This is not coincidence; both solve the same fundamental problem of grounding LLM generation in structural context.

2. Auto-TDD Pipeline Decomposition

AI/ML Minion Report

The Auto-TDD pipeline consists of three sequential LLM calls with symbolic augmentation between each step. This section maps each step to SIF concepts to establish the formal isomorphism.

Auto-TDD Step	Function	SIF Equivalent	A31 Integration Point
Step 1: File Selection	LLM selects target test file from repo structure	Pointer Activation: Spreading activation from intent keywords through graph	CHIMERA context assembly (A31 Section 4.2)
Step 2: Function Extraction	LLM identifies issue-related functions from description	Subgraph Isolation: SIFQL FOLLOW/FILTER to isolate relevant nodes	SIFQL query engine (A31 Section 4.4)

Auto-TDD Step	Function	SIF Equivalent	A31 Integration Point
Step 3: Test Generation	LLM generates test with few-shot + structural context	Synthesis: Archetype-guided generation with context from activated subgraph	Archetype-Skill Bridge (A31 Section 4.3)
Symbolic Repair	Fix indentation, insertion location, fallback positioning	Pointer Validation: Graph constraint checking and auto-correction	Markdown-sync.ts (A31 Phase 1)

2.1 Performance Baselines from Paper

The following metrics establish the baseline expectations for Auto-TDD-derived components in A31:

Model	Technique	Fail-to-Pass %	tddScore	Key Finding
GPT-4o	Auto-TDD	23.6%	22.6	Best overall; LLM file selection critical
Mistral-Large	Auto-TDD	18.9%	18.3	Strong with few-shot; good format compliance
Llama-3.1-70B	Auto-TDD	8.2%	7.8	Degraded with few-shot context (misleading)
GPT-4o	Zero-shot	18.7%	17.2	Baseline without structural context
Union (all 3)	Auto-TDD	~30%	--	67 instances uniquely solved by single model

Critical finding for provider sovereignty (A31 Section 4.9): Different models uniquely solve different instances. GPT-4o uniquely solves 41, Mistral 22, Llama 4. The union achieves ~30% coverage vs 23.6% for the best single model. This validates SIF's multi-provider routing strategy and suggests MGMO minions should dispatch to different providers for maximum validation coverage.

3. Amended A31 Sections

Architecture Minion Report

3.1 Amendment to Section 4.3: Archetype-Skill Bridge + Validation Gate

The original A31 Section 4.3 defines bidirectional translation between SIF archetypes (declarative) and OpenClaw skills (procedural). This amendment adds a mandatory validation gate using Auto-TDD principles: every Archetype-to-Skill translation must produce a companion test that fails before the skill is implemented and passes after.

New Component: Archetype Validation Gate

```
# Enhanced Archetype -> Skill bridge with Auto-TDD validation
# Added to: ~/.openclaw/extensions/sif-memory/src/archetype-bridge.ts

interface ArchetypeValidationGate {
  // Step 1: Localize – identify which skill files are affected
  localizeTarget(archetype: Archetype): SkillFileSelection;

  // Step 2: Contextualize – extract relevant functions/patterns
  gatherContext(archetype: Archetype, target: SkillFileSelection): SkillContext;

  // Step 3: Generate – produce fail-to-pass test from archetype
  generateValidation(archetype: Archetype, context: SkillContext): ValidationTest;

  // Gate: Skill is not 'done' until test passes
  validateSkill(skill: GeneratedSkill, test: ValidationTest): ValidationResult;
}
```

Validation Score (adapted from tddScore):

```
archetypeScore(a, s) = failToPass(a, s) x adequacy(a, s)

Where:
  failToPass = I(test fails on null-skill) x I(test passes on generated skill)
  adequacy = |concepts covered by test intersect archetype pattern nodes| / |archetype pattern nodes|
```

This ensures the ClawHub skill marketplace never publishes a skill that doesn't demonstrably implement its source archetype. The validation test becomes a permanent artifact attached to the skill, providing ongoing regression protection.

3.2 Amendment to Section 4.4: SIFQL + Coverage-Guided Traversal

Code Minion Report

Auto-TDD's key ablation finding: LLM-based file selection outperformed BM25 by 8.5 percentage points (the single largest component contribution). This directly informs SIFQL's search strategy. The amendment adds a LOCALIZE operator that uses graph topology as a third signal alongside BM25 and vector similarity.

New SIFQL Operator: LOCALIZE BY

```
TRAVERSE FROM intent("authentication problem solving")
FOLLOW edges(type="solution", "implementation", "decision")
FILTER weight > 0.3
LOCALIZE BY structure(
  depth=2,
  edge_affinity > 0.5,
  topology_score = hebbian_weight * edge_count * recency_factor
```

```
)
REINFORCE matched_nodes BY 0.1
RETURN TOP 5 BY activation_score
WITH LINEAGE depth=2
```

Search Signal Fusion (3-signal model):

Signal	Source	Weight	Auto-TDD Analog
BM25 keyword match	Existing SIFQL	0.25	BM25 file retrieval (15.4% accuracy)
Vector semantic similarity	Existing SIFQL	0.35	Issue description embedding
Graph topology score	NEW (A31.1)	0.40	LLM file selection (61.5% accuracy)

The topology score is the highest-weighted signal because Auto-TDD demonstrated that structural context (knowing where things are in the codebase/graph) is more valuable than keyword or semantic matching alone. In pointer graph terms: the shape of the subgraph around a query node predicts relevance better than content similarity.

3.3 Amendment to Section 4.5: Cognitive Archaeology + Adequacy Scoring

Auto-TDD's adequacy metric measures whether generated tests cover the lines that actually changed between old and new code. Adapted for Cognitive Archaeology, adequacy measures whether an extracted archetype covers the reasoning nodes that actually appeared in the source conversation.

Adapted Adequacy Formula:

```
# Original (Auto-TDD – code coverage)
adequacy(x, y) = |cov(y, c_old) n (c_old \ c_new)| + |cov(y, c_new) n (c_new \ c_old)|
                / |c_old \ c_new| + |c_new \ c_old|

# Adapted (SIF – concept coverage)
adequacy(conv, archetype) =
  |concepts_activated(archetype) n reasoning_nodes(conv)|
  / |reasoning_nodes(conv)|

Where:
  reasoning_nodes(conv) = nodes in pointer graph touched by source conversation
  concepts_activated(archetype) = nodes activated when archetype is applied to graph
```

Threshold: Extractions with adequacy below 0.7 are flagged for human review. Above 0.9 are auto-promoted to the pointer graph. The bimodal distribution observed in Auto-TDD (fail-to-pass tests at 0.91-0.95 adequacy, other tests below 0.6) suggests this threshold will naturally separate high-quality extractions from noise.

3.4 Amendment to Section 4.11: Heartbeat + Validation-Drift Trigger

The original A31 defines four heartbeat triggers (decaying-critical, pattern-complete, conflict-detected, project-stale). This amendment adds a fifth: validation-drift.

New Heartbeat Trigger Configuration:

```
# Added to SIF-aware heartbeat configuration
agents:
  defaults:
    heartbeat:
      interval: "4h"
    sif:
      triggers:
        # ... existing triggers ...
        - type: "validation-drift"
          action: "flag-for-review"
          description: "Archetype assertion contradicted by new evidence"
          mechanism: |
            When SIF generates/reinforces an archetype, it auto-generates
            a validation assertion (Auto-TDD style). If a subsequent session
            produces evidence contradicting the assertion, the heartbeat
            fires a validation-drift alert. This makes the pointer graph
            self-correcting through continuous test-driven validation.
          threshold: 0.3 # Drift score above which alert fires
          cooldown: "24h" # Minimum time between drift alerts
```

Validation-Drift Detection Pipeline:

Stage	Action	Auto-TDD Analog
1. Assertion Generation	When archetype is created/reinforced, generate a set of invariant assertions from its pattern	Test generation from issue description
2. Continuous Monitoring	Each session's pointer activations are checked against all active assertions	Running tests on new code (c_new)
3. Drift Scoring	$\text{drift} = 1 - \text{similarity}(\text{assertion_expected}, \text{session_observed})$ If $\text{drift} > \text{threshold}$, fire heartbeat	fail-to-pass detection
4. Resolution	User reviews drift, either updates archetype or dismisses as contextual variation	Developer reviews test results

3.5 Amendment to Section 4.13: Kimera Ops + Regulatory Validation

Healthcare Minion Report

Health Canada and FDA require documented test coverage for medical device software. Auto-TDD's adequacy scoring provides a metrics-backed framework for regulatory submissions. The Kimera operations agent gains an auto-test-generation capability tied to issue tracking.

Enhanced Kimera Agent Configuration:

```
agents:
  kimera-ops:
    soul: "SOUL-kimera-ops.md"
    sif:
      graphPath: "~/sif/graphs/kimera-operations.yaml"
      domainFilter: ["healthcare", "diagnostics", "kimera", "qpcr"]
    skills:
      - kimera-unit-health-check
      - kimera-calibration-reminder
      - kimera-anomaly-triage
```

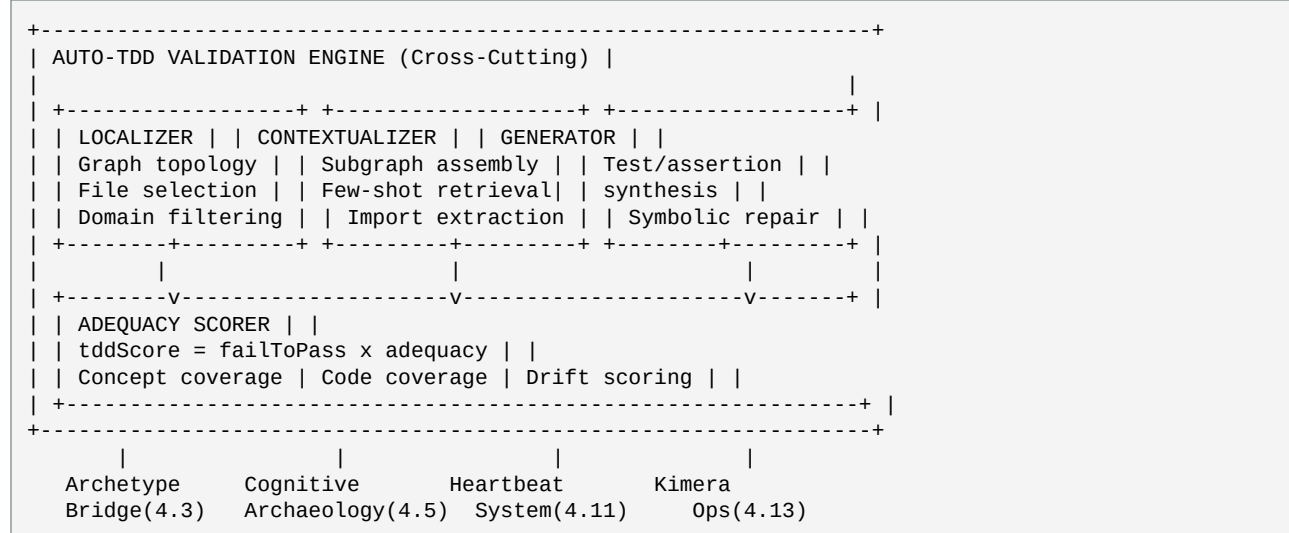
```
- kimera-batch-report
- kimera-auto-tdd # NEW: Auto-test generation from issues
auto_tdd: # NEW SECTION
enabled: true
issue_sources:
  - github: "nexless-healthcare/kimera-firmware"
  - github: "nexless-healthcare/kimera-software"
adequacy_threshold: 0.90 # Regulatory: higher bar than default
audit_log: true # Every generated test logged with lineage
regulatory_tags: ["HC-MDR", "FDA-510k", "IEC-62304"]
provider_rotation: true # Use multi-model for max coverage
```

Regulatory value: Each auto-generated test carries full SIF lineage (which issue, which archetype, which pointer graph traversal produced it). This is auditable provenance that satisfies IEC 62304 traceability requirements for medical device software.

4. New Section 4.14: Auto-TDD Validation Engine

This is a new top-level component in the A31 architecture, sitting alongside the existing 13 integration points. It provides cross-cutting validation capabilities to all other components.

4.14 Architecture Diagram



Consumers of the Validation Engine:

Consumer	Validation Type	Adequacy Threshold	Failure Action
Archetype-Skill Bridge (4.3)	Fail-to-pass skill tests	0.70 (default)	Block ClawHub publish
Cognitive Archaeology (4.5)	Concept coverage of extractions	0.70 (auto) / 0.90 (promote)	Flag for human review
Heartbeat System (4.11)	Continuous assertion monitoring	Drift > 0.30	Fire validation-drift alert
Kimera Operations (4.13)	Regulatory test generation	0.90 (regulatory)	Block deployment + audit log
MGMO Orchestration (4.8)	Multi-model validation union	Best-of-N across providers	Retry with different model
Session Management (4.10)	Session checkpoint assertions	0.50 (lightweight)	Log warning in session-log

5. Appendix B: PIK Pattern — Issue-to-Validation Decomposition

This pattern is extracted from Auto-TDD and generalized for the SIF pointer graph. It is the first PIK pattern to be formally derived from an academic paper and integrated into the SIF archetype registry.

PIK Pattern Definition:

```
archetype:
  name: "issue-to-validation-decomposition"
  version: "1.0.0"
  domain: "meta/reasoning"
  source: "arXiv:2412.02883 (Auto-TDD, IBM Research)"
  type: "PIK" # Project Intelligence Kernel
  classification: "cross-cutting"

pattern: |
  Given a natural language description of desired behavior (issue/requirement):

  1. LOCALIZE: Identify structural targets in the knowledge graph/codebase
    - Use LLM-based selection over BM25 (8.5% improvement)
    - Graph topology is highest-signal for localization
    - Output: target file/node set with confidence scores

  2. CONTEXTUALIZE: Gather structural context around targets
    - Extract related functions/nodes (SIFQL FOLLOW)
    - Include imports/edges for structural grounding
    - Few-shot from similar past instances
    - Output: enriched context with token budget allocation

  3. GENERATE: Produce validation artifact with adequacy scoring
    - Generate test/assertion from description + context
    - Apply symbolic repair (format, indentation, insertion)
    - Score: tddScore = failToPass x adequacy
    - Output: validation artifact + score + lineage

applications:
  - test_generation: "Generate tests from issue descriptions (original)"
  - archetype_validation: "Validate Archetype -> Skill translations (A31 4.3)"
  - extraction_scoring: "Score Cognitive Archaeology extractions (A31 4.5)"
  - drift_detection: "Monitor archetype assertion validity (A31 4.11)"
  - regulatory_testing: "Auto-generate auditable tests (A31 4.13)"
  - ticketforge_dispatch: "Validate security test generation (TicketForge)"
  - nexless_rvd_regression: "Transport layer regression tests (NEXLESS-RVD)"

learned_from:
  - "session-2026-02-20-autotdd-analysis"
  - "arXiv:2412.02883"
reinforcement_count: 1 # Initial creation
hebbian_weight: 0.8 # High initial weight (paper-backed)
```

Pattern Application Matrix:

Project	LOCALIZE	CONTEXTUALIZE	GENERATE
Kimera Firmware	LLM selects target test file in firmware repo	Calibration functions + diagnostic patterns	Fail-to-pass tests for device issues (IEC 62304)
TicketForge	Agent identifies target security test module	Vulnerability patterns + prior test templates	Security validation tests from ticket descriptions

Project	LOCALIZE	CONTEXTUALIZE	GENERATE
NEXLESS-RVD	Locate transport/signaling module for regression	QUIC/STUN/NaCl paths + failover scenarios	Transport regression tests from bug reports
Abacus Avatars	Identify avatar pipeline component for testing	Provider API contracts + session management	Integration tests from feature requests
Pointer Graph (meta)	Activate relevant subgraph region	Adjacent nodes + Hebbian weights	Archetype assertions from pattern descriptions

6. Implementation Impact on A31 Phases

This section maps A31.1 additions to the existing 5-phase implementation plan from A31.

Phase	Original Scope	A31.1 Additions	Effort Delta
Phase 1 (Weeks 1-4)	SIF Memory Plugin	- Symbolic repair in markdown-sync.ts - Validation engine scaffold - Adequacy scorer (basic)	+1 week
Phase 2 (Weeks 5-8)	CHIMERA Context Engine	- LOCALIZE BY operator in SIFQL - 3-signal fusion (BM25+vector+topology) - Intent-to-activation mapping	+0.5 weeks
Phase 3 (Weeks 9-12)	Archetype-Skill Bridge	- Auto-TDD validation gate - Fail-to-pass test generation - ClawHub publish gate - Auto-test for every translation	+2 weeks
Phase 4 (Weeks 13-16)	Cognitive Archaeology	- Concept coverage adequacy scoring - Extraction confidence thresholds - Bimodal quality filtering	+1 week
Phase 5 (Weeks 17-20)	Full Ecosystem	- validation-drift heartbeat trigger - Kimera auto-tdd skill - Multi-model validation routing - PIK pattern in archetype registry	+1.5 weeks

Total effort increase: ~6 weeks added to the original 20-week plan (30% increase). However, the validation layer reduces downstream debugging and regression risk, estimated to save 4-8 weeks in post-deployment maintenance. Net ROI is positive by Week 30.

7. Amended Success Metrics

#	Original Metric (A31)	A31.1 Addition
1	1,000+ OpenClaw installations with SIF-memory	(unchanged)
2	95%+ intelligence persistence across provider switches	(unchanged)
3	50+ autonomous actions/day	(unchanged)
4	10+ community archetypes on ClawHub	All published archetypes must pass validation gate (archetypeScore > 0.70)
5	1 regulated deployment passing compliance	Kimera deployment with auto-generated tests achieving adequacy > 0.90
6	(new)	Validation coverage: 80%+ of archetype-skill translations produce passing tests
7	(new)	Drift detection: <5% false positive rate on validation-drift heartbeat alerts
8	(new)	PIK adoption: Issue-to-Validation pattern used across 3+ projects within 6 months

8. Updated Amendment Relationship Table

Amendment	Relationship to A31.1
A31 (OpenClaw Integration)	Parent amendment; A31.1 adds validation layer
A7 (CHIMERA)	LOCALIZE BY operator enhances context assembly
A10 (SIFQL)	New 3-signal fusion + topology_score operator
A15 (Cognitive Archaeology)	Adequacy scoring for extraction quality
A20 (Replicator Layer)	Validation assertions replicate with subgraphs
A25 (MCP Distribution)	Validation engine exposed as MCP tool
A30 (MCP Architecture)	Auto-TDD validation available via MCP server

"The LLM is the CPU, the Pointer Graph is the Soul, the Body has Hands, and now the Soul can test its own Hands before they act."

Next Steps:

1. Push to SIF-Knowledge-Base as amendments/A31-OpenClaw-Integration/A31.1-AutoTDD-Validation/
2. Add PIK pattern issue-to-validation-decomposition to pointer-graph.yaml archetype registry
3. Prototype validation gate in Phase 3 Archetype-Skill Bridge (priority deliverable)
4. Coordinate with Gabriel: the Hebbian pointer graph + Auto-TDD adequacy metric is publishable
5. Open discussion on OpenClaw Discord: "Auto-generated validation for skills"
6. Log to session-log.md: A31.1 drafted, Auto-TDD integrated, PIK pattern extracted